Setting up the data repository

Basket Reporting · what to click, in order, so that the app can write and the Notion integration can read · v260813-01

What you are building

Three parties touch one file. The **app** on the iPads writes it, **GitHub** holds it, and the **Notion integration** reads it. Each side needs its own token, and the two tokens deliberately do not have the same rights: the crew's token may write, the integration's token may only read. Nothing else is shared.

	Repository	Who touches it	Token
1	`bwicki/gb_notion_frontend`	GitHub Pages serves the app from it. Nobody writes flight data here.	none
2	`bwicki/gb_flight_data`	The iPads write the flight table. Notion reads it.	two, see step 3 and 5

> **Keep them apart.** Every entry is two commits. If they land in the repository that GitHub Pages builds from, every entry triggers a Pages rebuild, and Pages throttles at roughly ten builds an hour. A flight with a hundred entries would jam the queue and could stop the app itself from loading.

Step 1 — create the data repository

On github.com, signed in as the owner of the flight data:

1. Top right **+** → **New repository**
2. Owner: your account. Repository name: `gb_flight_data`
3. Visibility: **Private** — the position, track and ballast history of every flight would otherwise be readable by anyone who finds the URL. Notion can read a private repository through the API without any trouble.
4. Tick **Add a README file**
5. Create repository

> **The README tick is not cosmetic.** A repository with no commits has no branch yet, and the GitHub API cannot write into a branch that does not exist. Without it the first entry fails with a 404 and the cause is not obvious from the app.

Do not create a `data` folder. The app creates it, and the files inside it, on its first successful post.

Step 2 — note the exact name

Everything below refers to the repository as **owner/name**, exactly as it appears in the address bar. Case matters.

```
https://github.com/bwicki/gb_flight_data   →   bwicki/gb_flight_data
```

Step 3 — the writing token, for the iPads

Settings → Developer settings → Personal access tokens → **Fine-grained tokens** → **Generate new token**

Field	What to set	Why
Token name	`basket-write-2026`	So you recognise it when revoking.
Expiration	shortly after the season	A token that expires is one you cannot forget.

Resource owner	your account	The account that owns the data repository.
Repository access	Only select repositories → gb_flight_data	Nothing else can be touched with it.
Repository permissions → Contents	Read and write	The only permission the app needs.
everything else	No access	Metadata switches itself to read-only. That is expected.

Generate token, then copy it at once — GitHub shows it exactly once. It begins with `github_pat_`. If you lose it, revoke it and make a new one; there is no way to read it back.

Step 4 — put it into the app

On the **master** device, the one that answered the master question with 5678:

1. Menu → **Settings** → **Unlock settings**, password **1234**
2. Fill in the flight ID, the callsign and both pilots
3. Open **GitHub connection** and enter:

Repository	`bwicki/gb_flight_data`
Branch	`main`
Folder	`data`
Fine-grained token	the token from step 3

4. Check connection — it must name the repository back to you. If it says *Repository not found, or the token has no access*, the name is misspelt or the token was scoped to the wrong repository.
5. Save settings

Step 5 — the reading token, for Notion

Make a **second** token. Do not reuse the one on the iPads: a third party's integration should not be able to write into the flight log, and a token you can revoke separately is one you can revoke without grounding the crew.

Field	What to set
Token name	`notion-read-2026`
Repository access	Only select repositories → gb_flight_data
Repository permissions → Contents	Read-only
everything else	No access

Hand this token to whoever built the Notion integration, over a channel you would send a password over. Tell them it is read-only and scoped to one repository, so it cannot damage anything.

Step 6 — what the Notion side has to call

One request, repeated on a timer. Substitute the flight ID; the file is named after it, with anything that is not a letter, digit, hyphen or underscore replaced by a hyphen.

```
GET https://api.github.com/repos/bwicki/gb_flight_data/contents/data/GB-2026-01.json
Authorization: Bearer <notion-read token>
Accept: application/vnd.github.raw
X-GitHub-Api-Version: 2022-11-28
```

The Accept header matters. With `application/vnd.github.raw` the response body is the JSON file itself. Without it, GitHub returns an envelope with the file base64-encoded in a `content` field, which then has to be decoded.

Two more files worth polling

`data/<flight>.json`	the table — every row, sorted by time
`data/<flight>.deleted.json`	ids that were deleted; archive or hide those rows in Notion
`data/_flights.json`	the index of closed flights, written when a new flight is begun

Import rule

Upsert on `id`. Never append. The file always holds the complete set of rows, not a delta. An integration that appends on each poll will multiply the table on the second poll.

Rate limit: 5000 requests per hour per token. A thirty-second poll is 120 an hour. If the integration sends `If-None-Match` with the ETag of the previous response, unchanged files answer 304 and cost nothing at all.

Step 7 — the live test, in this order

#	Do this	What must happen
1	On the master: Menu → Settings → GitHub connection → Check connection	A message naming `bwicki/gb_flight_data`.
2	Post one Ballast entry, for example a 5 kg drop	The post button turns **pale green** and reads *Posted to CC Notion*. Pale yellow means no connection; pale red names the reason.
3	Open the repository on github.com	`data/<flight>.json` and `.csv` exist, with one row.
4	Menu → Log	The entry carries a **green double check** at its right edge.
5	Let the Notion integration poll once	One row appears in Notion, with the flight, the time, the reporter and the position.
6	Post a second entry, then poll again	Two rows in Notion — not three. Three means it is appending instead of upserting.
7	Turn off the iPad's connection, post a third entry	The button turns **pale yellow**, the log shows a straw single check, the header counts one pending.
8	Turn the connection back on and wait five seconds	The check turns green on its own, and the row reaches Notion on the next poll.
9	In the log, open an entry and delete it, confirming twice	Its id appears in `data/<flight>.deleted.json`.
10	Delete the test rows in Notion and press *Clear entries on this device*, then begin the real flight	An empty table for the flight that counts.

When something does not arrive

Symptom	Cause
Header says *no repository*	Repository or token missing in the settings of that device.
First post fails with 404	The data repository has no branch. Add a README through the web interface.
403 on posting	The token lacks Contents: Read and write, or does not include this repository.
Entries stay straw	No connection, or the token has expired. *Send now* in the menu shows the real error.
Red cross in the log	An attempt was refused. Check the token's expiry date first.
Notion shows duplicates	It is appending instead of upserting on `id`.
Notion is minutes behind	It is polling raw.githubusercontent.com, which sits behind a five-minute cache. Use the API.
Notion sees nothing at all	Read token scoped to the wrong repository, or the flight ID in the path is not the current one.

Both tokens can be revoked at any time under Settings → Developer settings → Personal access tokens. Revoking the writing token stops the iPads; revoking the reading token stops Notion. Nothing already written is affected either way.